

THE UNIVERSITY OF DODOMA



**COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

FINAL YEAR PROJECT PROGRESS REPORT 1

**COURSE NAME: CYBER SECURITY AND DIGITAL FORENSICS ENGINEERING
PROJECT I**

COURSE CODE: CS 418

SEMESTER: 1

REPORT AIM: PROGRESS

ACADEMIC YEAR: 2025/2026

**TITLE: AN AUTOMATED DIGITAL TIMESTAMP FORGEY DETECTION TOOL
USING TIMELINE CORRELATION AND EVIDENCE SCORING IN WINDOWS NTFS**

GROUP 15 MEMBERS

S/N	FULL NAME	REGISTRATION NUMBER	PROGRAM
1	SARAFINA W MGANI	T22-03-01712	Bsc. CSDFE
2	ELIFURAHA S KIANGI	T22-03-08062	Bsc. CSDFE
3	MATHIAS M GEORGE	T22-03-07128	Bsc. CSDFE
4	MUGHUTARI S MBWANA	T22-03-10581	Bsc. CSDFE
5	NOEL D KWEKA	T22-03-07117	Bsc. CSDFE

NAME OF SUPERVISORS

SIGNATURE

Mr Ona Nixon

.....

Table of Contents

<i>List of Figures</i>	ii
<i>List of Tables</i>	ii
CHAPTER 01: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Project Significance	2
1.5 Potential Customers, Competitors, and Market Analysis	3
1.6 Scope of the Study	3
CHAPTER 02: LITERATURE REVIEW	4
2.1 Review of Practical Tools and Their Use	4
2.1.1 Plaso/Log2Timeline	4
2.1.2 MFTECmd (Master File Table Extraction Command)	4
2.1.3 nTimetools	5
2.1.4 TimeStomp Detection Scripts	5
2.1.5 Autopsy and The Sleuth Kit (TSK)	5
2.2. Research Gap	6
2.2.1 Manual Correlation and Expert Dependence	6
2.2.2 Limited Artifact Integration	6
2.2.3 Absence of Automated Evidence Scoring	6
2.2.4 Need for Automated Multi-Source Correlation	6
CHAPTER 03: METHODOLOGY	7
3.1 Introduction	7
3.2 Project Approach	7
3.3 Achieving the specific objectives	8
CHAPTER 04: TIME FRAME AND BUDGET	9
4.1 Project Time Frame (January – June)	9
4.2 Budget Estimate (TZS)	10
CHAPTER 05: SYSTEM ARCHITECTURE AND DESIGN	11
5.1 System design	11
5.2 Functional and non-Functional requirements	11
5.3 System Architecture	12
5.3 UML Diagrams	13
i. Use Case Diagram	13
ii. Sequence Diagram	14
iii. Data Flow Diagram	15
iv. System Flowchart	17
REFERENCES	19

List of Figures

<i>Figure 1 : Project Timeline</i>	<i>9</i>
<i>Figure 2 : System Architecture</i>	<i>13</i>
<i>Figure 3 : Use Case Diagram</i>	<i>14</i>
<i>Figure 4 : Sequence diagram</i>	<i>15</i>
<i>Figure 5 :dfd Level 0</i>	<i>16</i>
<i>Figure 6 : dfd Level 01.....</i>	<i>16</i>
<i>Figure 7 : General Flow chart</i>	<i>17</i>
<i>Figure 8 : Forgery Flow Chart</i>	<i>18</i>

List of Tables

Table 1 : Budget Estimation Table.....	10
--	----

CHAPTER 01: INTRODUCTION

1.1 Background

Today, almost everything we do is recorded by computers. When we write a document, take a photo, send an email, or download a file, the computer automatically records the time when it happened. These times, such as when a file was created or modified or accessed, help us understand what happened first and what happened later. Shortly, time helps us tell the story of events on a computer.

In many situations such as crime investigations, court cases, school disputes, and business records, these time records are very important. For example, a file might be used to prove when a report was written, when a contract was signed, or when a virus was installed. If the time of a file is masted, then the events around it can also be trusted.

However, some people try to cheat this system. They use special software to change the time of a file so that it looks older or newer than it really is. This is called digital timestamp forgery. For example, a hacker may create a file today but change its date to make it look like it was created years ago. This can be used to hide crimes, create fake evidence, or avoid responsibility.

The problem is that when someone changes a file's time, the computer still keeps hidden records of what really happened. These records are stored deep inside the computer system and include things like file system logs, system journals, and backup copies. These hidden records are much harder to change, and they usually tell the true story. Unfortunately, reading and understanding these hidden records is very difficult and requires expert knowledge and a lot of manual work.

Most existing forensic tools only show raw data and long lists of times. Investigators must spend many hours comparing them to decide whether a file is real or fake. Most tools do not automatically tell whether, "This file is forged" or "This file is genuine."

This project "An automated digital forensic tool for detecting timestamp forgery in Windows NTFS disk image using timeline correlation and evidence scoring" is designed to solve this problem. Our tool will collect time information from many different hidden sources in the computer, compare them automatically, and calculate how likely it is that a file's timestamp has been changed. In simple terms, our tool will act like a digital detective that checks all the clues and gives a clear answer about whether a file is telling the truth about its time.

1.2 Problem Statement

Digital investigations rely on file timestamps to reconstruct what happened on a computer and when it happened. These timestamps are often used as digital evidence in cybercrime cases, legal disputes, and organizational audits. However, attackers can easily change file timestamps, making false timelines that hide real activities.

Although computers keep many hidden records, such as system logs and file system journals, that contain the true history of file activities, these records are scattered across different sources. Current forensic tools can extract this information, but they usually present it as separate timelines

or raw data. Investigators must manually compare these timelines to decide whether a file's timestamp is genuine or manipulated

This manual process is slow, complex, and depends heavily on human judgment. There is no consistent way to measure how strong or weak the evidence of timestamp manipulation is. As a result, different investigators may reach different conclusions about the same file.

Because of this there is a need for a tool that can automatically correlate timestamps from multiple sources and provide a clear, objective measure of how likely it is that a file's timestamp has been Forged. Without such a tool, timestamp forgery remains difficult to detect reliably, and digital evidence remains uncertain.

1.3 Objectives

Main Objective

To develop and implement an automated digital forensic tool for detecting file timestamp forgery in Windows NTFS file systems using timeline correlation and evidence scoring techniques.

Specific Objectives

- i. To understand how file timestamps work in the Windows NTFS file system, including created, modified, accessed, and changed times.
- ii. To identify a common method used to manipulate or forge NTFS timestamps in Windows environments.
- iii. To develop a web-based automated tool that extracts and processes timestamp artifacts from NTFS disk images to construct a unified forensic timeline.
- iv. To analyze abnormal or inconsistent timestamp patterns through cross-artifact timeline correlation evidence-based scoring techniques.
- v. To generate a structured and comprehensive forensic report that presents the reconstructed timeline, detected timestamp anomalies, and corresponding evidence scores in a clear, consistent, and interpretable format.

1.4 Project Significance

This project will provide several important benefits:

- ❖ Automates timestamp forgery detection, saving investigators significant time and reducing reliance on manual, error-prone analysis.
- ❖ Improves accuracy and objectivity in digital forensic investigations by providing consistent, evidence-based forgery probability scores.
- ❖ Strengthens the reliability of digital evidence in courts, law enforcement, and organizational audits, helping prevent miscarriages of justice.
- ❖ Supports cybersecurity education and capacity building in Tanzania, where cybercrime is rising but forensic resources and expertise are limited.
- ❖ Serves as a practical tool for police cyber units, academic institutions, and private organizations while laying a foundation for future enhancements in digital forensics.

1.5 Potential Customers, Competitors, and Market Analysis

Potential Customers

- ❖ Law enforcement cybercrime units (eg., Tanzania Police Force Cybercrime Unit)
- ❖ Government regulatory and intelligence agencies
- ❖ Private forensic laboratories and consultancies
- ❖ Corporate information security teams (banking, telecom, critical infrastructure)
- ❖ Legal firms specializing in cybercrime and e-discovery
- ❖ Universities and cybersecurity training institutions

Competitors

- ❖ Commercial Tools: Cellebrite UFED, Magnet AXIOM, OpenText EnCase, X-Ways Forensics
- ❖ Open-Source Tools: Autopsy/The Sleuth Kit, Plaso (Log2Timeline), MFTECmd

Market Analysis The global digital forensics market is projected to surpass USD 15 billion by 2027 driven by rising cybercrime, regulatory compliance, and data protection laws (e.g., Tanzania Personal Data Protection Act 2022). Existing tools lack automated timestamp forgery detection, creating a niche opportunity. This project addresses that gap by offering a specialized, cost-effective solution for NTFS timestamp forgery detection.

1.6 Scope of the Study

This project focuses on detecting timestamp forgery in Windows NTFS especial Scaling with disk image and it will: -

- ❖ Analyze forensic artifacts: MACB metadata, \$MFT (SSTANDARD_INFORMATION, SFILE NAME), \$UsnJml, \$LogFile, Registry, Event Logs, Volume Shadow Copies.
- ❖ Support disk image analysis via a web-based interface with visualization and reporting.
- ❖ Provide forgery probability scoring and automated correlation.

Exclusions: Other file systems (ext4, APFS), non-Windows OS (Linux, macOS, mobile), network/cloud timestamp manipulation, blockchain timestamps, and advanced anti-forensic techniques beyond common timestomping. Machine learning models are also excluded.

CHAPTER 02: LITERATURE REVIEW

2.1 Review of Practical Tools and Their Use

This section moves from general ideas to the actual tools used in digital forensics. By looking at the software used in the industry today, we can see how the concepts of finding data and building timelines work in real-life situations.

The success of an investigation often depends on how well these tools can extract and organize different pieces of digital evidence into a clear, timed list. The following sections look at the specific features, limitations, and reliability of the tools used in this study.

2.1.1 Plaso/Log2Timeline

Plaso, also known as Log2Timeline, is a widely used open-source forensic tool that collects timestamps from multiple system artifacts and creates a comprehensive super timeline. It can extract data from the MFT, SUsnJml, Event Logs, browser histories, and more.

Practical contributions:

- ❖ Demonstrates how timeline correlation works across multiple sources.
- ❖ Handles large datasets automatically, reducing manual effort.

Limitations:

- ❖ Does not provide a scoring mechanism to indicate the likelihood of timestamp forgery.
- ❖ Investigators must manually interpret inconsistencies.

2.1.2 MFTECmd (Master File Table Extraction Command)

MFTECmd is a high-speed command-line tool designed for extracting and analyzing the Master File Table (MFT) from NTFS file systems. It allows investigators to convert raw MFT data into readable formats like CSV or JSON, making it easier to track file creation, modification, and deletion events. By focusing on the MFT, this tool provides a look at the file system's history, even for files that have been recently removed.

Practical contributions:

- ❖ Allows detailed reconstruction of file activity, including creation, modification, and access times.
- ❖ Focuses primarily on MFT entries, providing high granularity for NTFS-based investigations.

Limitation

- ❖ Lack correlation with other artifacts such as shadow copies, registry entries, or event logs
- ❖ Manual comparison is often required to detect forgery.

2.1.3 nTimetools

nTimetool is forensic suite specifically engineered to detect timestamp anomalies that may indicate user interference. It functions by comparing the standard "MACB" (Modified, Accessed, and Birth) times against other system-generated logs to find inconsistencies. This comparison is vital for verifying whether a file's timestamp has been manipulated to hide activity or create a false alibi.

Practical contributions

- ❖ Highlight timestamp anomalies automatically.
- ❖ Provides pattern-based detection of suspicious changes.

Limitations

- ❖ Covers fewer artifact sources than other tools.
- ❖ May produce false positives without cross-checking multiple sources.

2.1.4 TimeStomp Detection Scripts

TimeStomp detection scripts are specialized forensic scripts used to identify deliberate timestamp Tampering, often referred to as "timestomping." These scripts work by cross-referencing file system timestamps in the MFT with secondary evidence found in the \$UsnJrnl (Update Sequence Number Journal) and Windows Event Logs. If a file's metadata does not match these independent system records, the script flags the entry as potentially altered.

Practical contributions

- ❖ Detects direct evidence of timestomping attempts
- ❖ Useful in live investigations to identify tampered timestamps quickly.

Limitations

- ❖ Typically limited in scope
- ❖ Requires manual verification and lacks standardized quantitative scoring.

2.1.5 Autopsy and The Sleuth Kit (TSK)

Autopsy is a graphical user interface (GUI) that simplifies complex forensic tasks, while The Sleuth Kit (TSK) serves as the powerful command-line engine that handles the data processing. Together, they provide an end-to-end solution for extracting digital artifacts and reconstructing system timelines. This combination is widely used to analyze disk images, allowing investigators to visualize how data moved through a system over a specific period.

Practical contributions:

- ❖ Extracts timestamps from multiple sources, including MFT, registry entries, shadow copies, and system logs.
- ❖ Supports timeline reconstruction across different file systems.
- ❖ Handles large disk images efficiently.

Limitations:

- ❖ Produces raw timelines without automated correlation of inconsistencies.
- ❖ No evidence scoring system; investigators interpret potential forgery manually.
- ❖ Large datasets still require significant expertise to analyze efficiently.

Why These Practical Works Matter

These practical tools demonstrate that timestamp forgery is a real, encountered problem in Forensic investigations. They show how multiple artifacts can be collected and timelines reconstructed, but also highlight key gaps:

- ❖ Existing tools largely rely on manual interpretation and expert knowledge.
- ❖ Correlation across multiple artifacts is limited or absent in most tools.
- ❖ Evidence scoring or quantitative assessment of forgery likelihood is not implemented.

These limitations justify the need for a system that automates multi-source correlation and produces a clear evidence score, enabling investigators to detect timestamp forgery efficiently and reliably.

2.2. Research Gap

2.2.1 Manual Correlation and Expert Dependence

Although existing forensic tools such as Plaso, MFTECmd, nTimetools, TimeStomp scripts, Autopsy, and The Sleuth Kit are effective in collecting timestamps and reconstructing timelines, they often require manual correlation and expert interpretation. This process is time-consuming and prone to human error, making it difficult for investigators to quickly and accurately identify timestamp forgery.

2.2.2 Limited Artifact Integration

Most current tools focus on a limited set of artifacts, such as MFT entries, system logs, or shadow copies, and rarely integrate all available sources like registry entries, event logs, application histories, and thumbnails. This lack of comprehensive cross-artifact correlation can reduce detection reliability, especially when attackers manipulate multiple artifacts to hide tampering.

2.2.3 Absence of Automated Evidence Scoring

Existing forensic solutions may flag anomalies but do not provide a quantitative measure or evidence score to indicate how likely a file's timestamps have been tampered with. Investigators must rely on subjective judgment, which can lead to inconsistencies and overlooked forgeries.

2.2.4 Need for Automated Multi-Source Correlation

These gaps highlight the need for a system that automatically collects timestamps from multiple artifacts, correlates them, and generates an evidence score. Such a system would improve accuracy, reduce reliance on manual analysis, and provide a standardized and reliable method for detecting timestamp forgery.

CHAPTER 03: METHODOLOGY

3.1 Introduction

This chapter describes the methodology to be used in the design and development of the Automated Digital Forensic Tool for Detecting File Timestamp Forgery in Windows NTFS file systems. The methodology outlines the systematic procedures to be followed in order to achieve the objectives of the project. It explains how file metadata will be acquired, processed, analyzed, and evaluated to identify inconsistencies in file timestamps that may indicate manipulation or forgery.

The proposed tool will follow a structured approach that includes system analysis to identify both functional and non-functional requirements of the forensic tool, data acquisition from NTFS file systems, extraction of file timestamps (created, modified, accessed, and entry modified times), and timeline reconstruction for forensic analysis. The methodology further incorporates timeline correlation techniques and evidence scoring mechanisms to assess the likelihood of timestamp tampering. In addition, a report generation component is included to automatically summarize findings and present forensic results in a structured and interpretable format for investigators.

Through this approach, the system aims to enhance digital forensic investigations by providing a reliable and automated mechanism for detecting suspicious timestamp alterations and supporting evidence-based conclusions.

3.2 Project Approach

This project will adopt the Agile system development methodology to guide the design and development of the Automated Digital Forensic Tool for detecting file timestamp forgery in Windows NTFS file systems. Agile is selected because it supports iterative development, flexibility, and continuous improvement throughout the project lifecycle.

The development process will be divided into small iterative cycles (sprints), where each sprint focuses on implementing and testing a specific module of the system. The initial phase will involve requirements analysis to identify key functionalities such as NTFS metadata extraction, timestamp retrieval, timeline reconstruction, evidence scoring, and report generation.

The system will then be developed incrementally, starting with core features such as file metadata extraction and timestamp analysis, followed by more advanced components such as timeline correlation and forgery detection mechanisms. The final iterations will focus on integrating all modules and implementing automated report generation.

After each sprint, the system will be tested and evaluated to identify errors and improve performance. Continuous feedback will be used to refine the system, ensuring accuracy, reliability, and effectiveness in detecting timestamp forgery. This Agile approach ensures the system evolves progressively while meeting the project objectives.

3.3 Achieving the specific objectives.

The specific objectives of this project will be achieved through the Agile methodology by dividing the work into iterative sprints, where each objective is addressed progressively through development, testing, and continuous improvement.

i. To understand how file timestamps work in the Windows NTFS file system

This objective will be achieved during the initial sprints through literature review and system analysis. Detailed study of NTFS timestamp attributes (created, modified, accessed, and changed times) will be conducted, and findings will guide the design of the metadata extraction module.

ii. To identify common methods used to manipulate or forge NTFS timestamps

This will be addressed alongside the first objective through research and experimental analysis. Different timestamp manipulation techniques will be studied and tested in controlled environments, and the results will inform the development of detection strategies within the system.

iii. To develop a web-based automated tool for extracting and processing timestamp artifacts

This objective will be achieved through iterative development sprints. Early iterations will focus on building the core functionalities such as NTFS data extraction and timestamp parsing, while subsequent sprints will implement timeline construction and integration into a web-based interface.

iv. To analyze abnormal or inconsistent timestamp patterns

This will be achieved in later sprints by developing and integrating timeline correlation and evidence-based scoring algorithms. The tool will analyze relationships between different timestamp artifacts to identify inconsistencies, with continuous testing to improve detection accuracy.

v. To generate a structured and comprehensive forensic report

This objective will be implemented in the final sprints by designing and integrating a report generation module. The module will compile the reconstructed timeline, detected anomalies, and evidence scores into a clear and interpretable format. Feedback from testing phases will be used to refine the report for usability and clarity.

Through Agile methodology, each objective is incrementally developed, tested, and improved, ensuring the final system is reliable, accurate, and aligned with the project goals.

CHAPTER 04: TIME FRAME AND BUDGET

4.1 Project Time Frame (January – June)

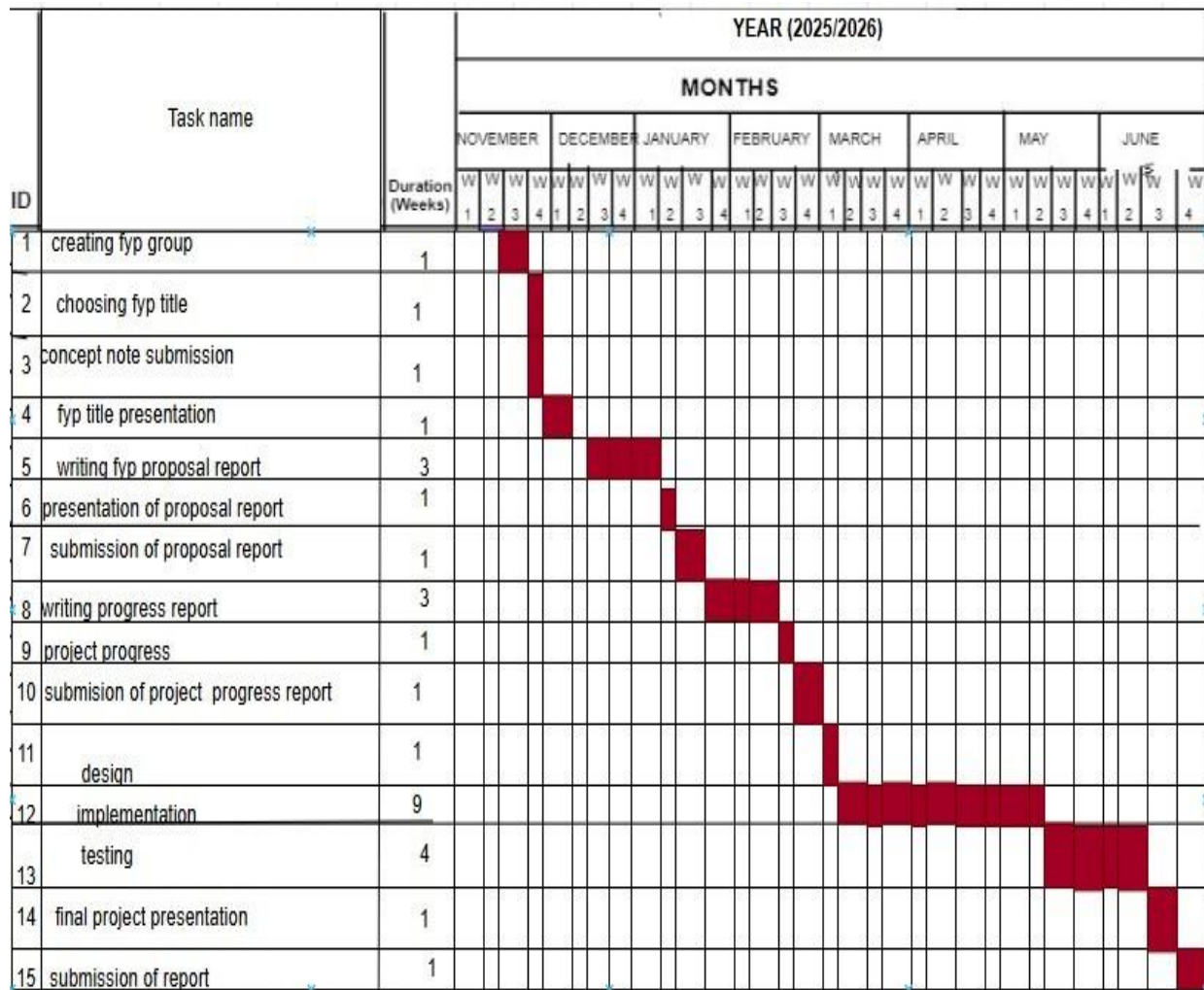


Figure 1 : Project Timeline

4.2 Budget Estimate (TZS)

Table 1 : Budget Estimation Table

Item	Description	Estimated Cost
High Speed Internet	Research, Repositories, Tool Downloads (6Months)	500,000
External Hard Drive (2TB)	Storage of forensic disk image and backups	400,000
Printing and Binding	Proposal, progress reports, final thesis	250,000
Stationery & Misc	Note books, USB Drives, Transports, Supervisor Meeting	200,000
Software/Hardware Buffer	Visualization Software or Minor hardware upgrades	150,000
Unforeseen Expenses	Buffer for unexpected costs	200,000
Total Estimated Budget		1,700,000/=

Note:

- ❖ Core development tools (Python, forensic libraries, web frameworks) are free.
- ❖ Costs are conservatively estimated to ensure sustainability.
- ❖ The budget will be managed and collected with regular reviews.

CHAPTER 05: SYSTEM ARCHITECTURE AND DESIGN

5.1 System design

System design is a key phase in the software development lifecycle that transforms system requirements into a structured plan for implementation. It bridges the gap between what the system should do and how it will function.

This chapter presents the design of the Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery, including system architecture, main components, and data flow for extracting and analyzing timestamp artifacts.

The design focuses on functional requirements, non-functional requirements, module design, and UML diagrams, which are explained in the following sections.

5.2 Functional and non-Functional requirements

Functional Requirements

1. The tool shall load and validate a Windows NTFS disk image by verifying its format, integrity, and file system structure before analysis.
2. The tool shall extract MACB timestamps and relevant forensic artifacts from NTFS sources, including \$MFT, \$LogFile, \$UsnJrnl, Windows Registry, Event Logs, and Volume Shadow Copies.
3. The tool shall normalize all extracted timestamps into a single standardized time format (UTC).
4. The tool shall store all extracted forensic data and associated metadata in a structured database for efficient analysis and querying.
5. The tool shall reconstruct a chronological timeline of file system activity based on correlated timestamp data.
6. The tool shall compare timestamps across multiple forensic artifacts to identify inconsistencies and detect possible timestamp manipulation (timestamping).
7. The tool shall assign reliability scores to different forensic artifacts based on their resistance to tampering.
8. The tool shall compute a forgery likelihood score for each file based on detected inconsistencies and artifact reliability.
9. The tool shall provide an interactive timeline visualization with filtering options by file name, time range, and artifact type.
10. The tool shall classify files as either genuine or forged based on the computed forgery likelihood score.
11. The tool shall generate automated forensic reports in PDF and/or HTML format, including analysis results, timelines, and conclusions.

12. The tool shall automate the entire forensic workflow from disk image loading to final report generation with minimal user intervention.

Non-Functional Requirements

1. The tool shall process NTFS disk image analysis within a reasonable time depending on the size of the image.
2. The tool shall efficiently handle large forensic disk images without significant performance degradation.
3. The tool shall accurately extract and normalize timestamps from NTFS forensic artifacts.
4. The tool shall minimize false positives and false negatives when detecting timestamp manipulation.
5. The tool shall produce consistent and repeatable results when analyzing the same disk image under identical conditions.
6. The tool shall provide a user-friendly interface that allows forensic investigators to easily operate and interpret results.
7. The tool shall present analysis results in a clear format, including timelines, summaries, and forensic findings.
8. The tool shall ensure that the original disk image remains unmodified during analysis to preserve evidence integrity.
9. The tool shall be modular to allow easy maintenance, debugging, and future feature enhancements, including additional forensic artifacts.
10. The tool shall operate reliably and recover gracefully from errors without loss of processed forensic data

5.3 System Architecture

The system architecture defines the main components of the **Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery** and the interactions among these components. It outlines how the system processes NTFS disk images, extracts timestamp artifacts, performs analysis using timeline correlation and evidence scoring, and generates forensic reports.

The following illustrates the system architecture for the proposed digital forensic tool.

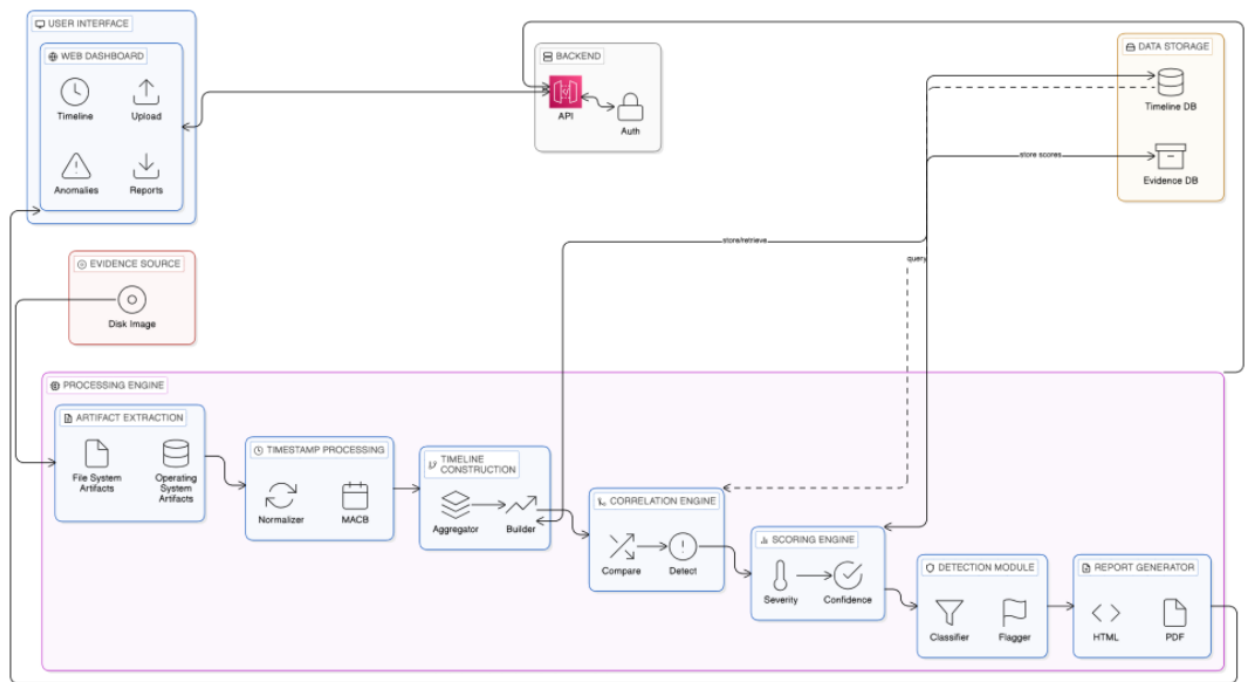


Figure 2 : System Architecture

5.3 UML Diagrams

Unified Modeling Language (UML) is a standard modeling language used in software engineering to visually represent system design. It helps in specifying, visualizing, and documenting the components and behavior of a system. UML diagrams simplify complex systems and improve communication among stakeholders.

The following are the UML diagrams for the **Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery**.

i. Use Case Diagram

A use case diagram is a UML behavioral diagram that represents the functional requirements of a system by showing interactions between users (actors) and system functionalities (use cases). It helps to clearly define what the system should do and identify its boundaries.

The following is the use case diagram for the Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery.

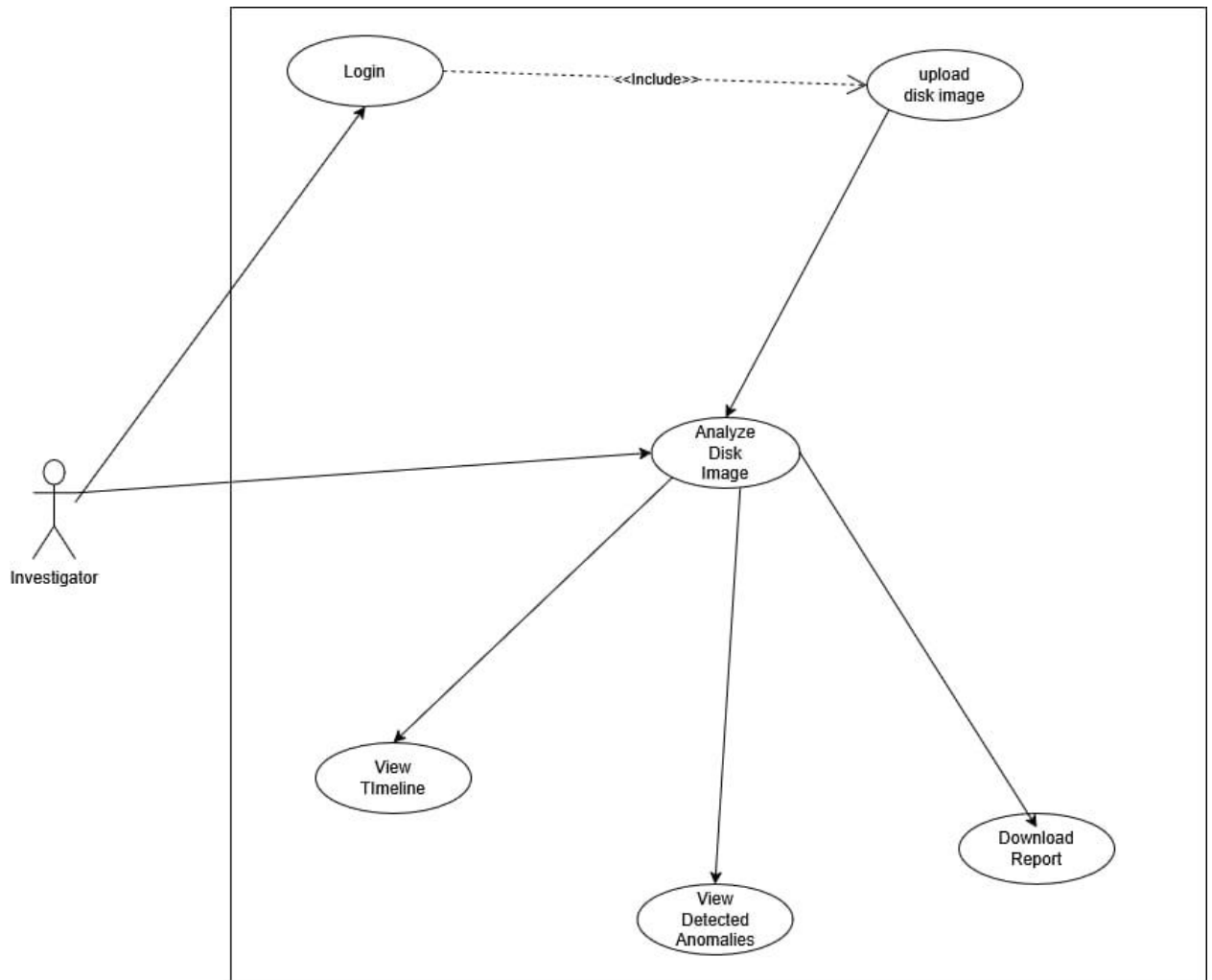


Figure 3 : Use Case Diagram

ii. Sequence Diagram

A sequence diagram illustrates the time-based interaction between system components, showing how processes communicate and the order in which messages are exchanged. It helps in understanding the detailed flow of system operations during a specific scenario.

For the **Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery**, the sequence diagram focuses on the process of extracting timestamp data, analyzing it using timeline correlation and evidence scoring, and generating the final forensic report.

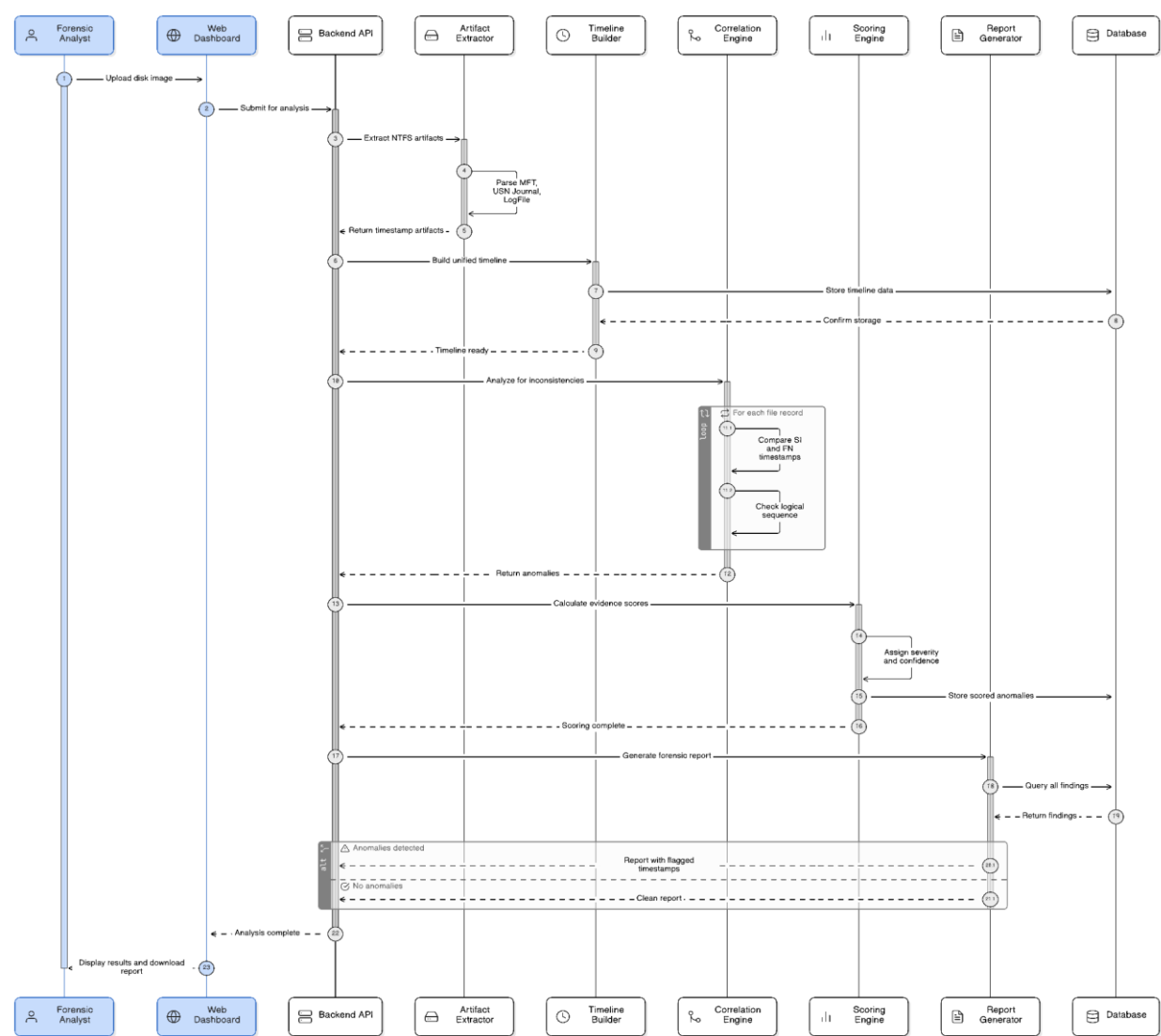


Figure 4 : Sequence diagram

iii. Data Flow Diagram

Data Flow Diagrams (DFDs) illustrate how data moves through a system, showing processes, data stores, and external entities. They focus on how data is input, processed, stored, and output, making them useful for understanding the system from a data perspective.

For the **Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery**, the DFD shows how NTFS disk images are input, how timestamp data is extracted and analyzed, and how results are stored and presented in forensic reports. A context diagram is also provided to give a high-level view of the entire system and its interaction with external entities.



Figure 5 :dfd Level 0

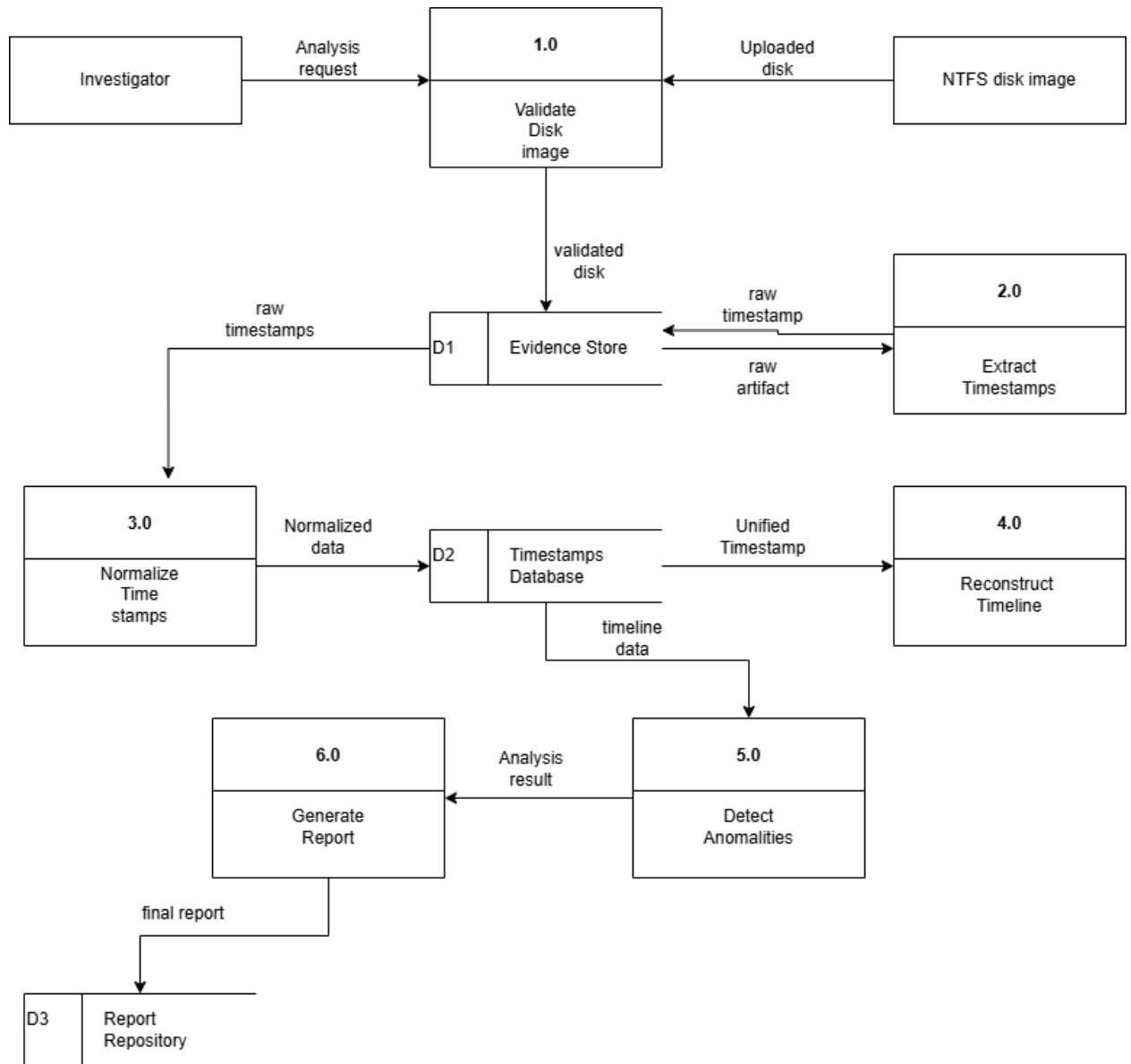


Figure 6 : dfd Level 01

iv. System Flowchart

The system flowchart represents the logical steps and decision points of the **Automated Digital Forensic Tool for Detecting NTFS Timestamp Forgery**. It illustrates the step-by-step process from input of NTFS disk images, extraction of timestamp data, analysis using timeline correlation and evidence scoring, to the generation of forensic reports.

The flowchart helps developers understand the overall logic of the system, including different processing paths and possible outcomes.

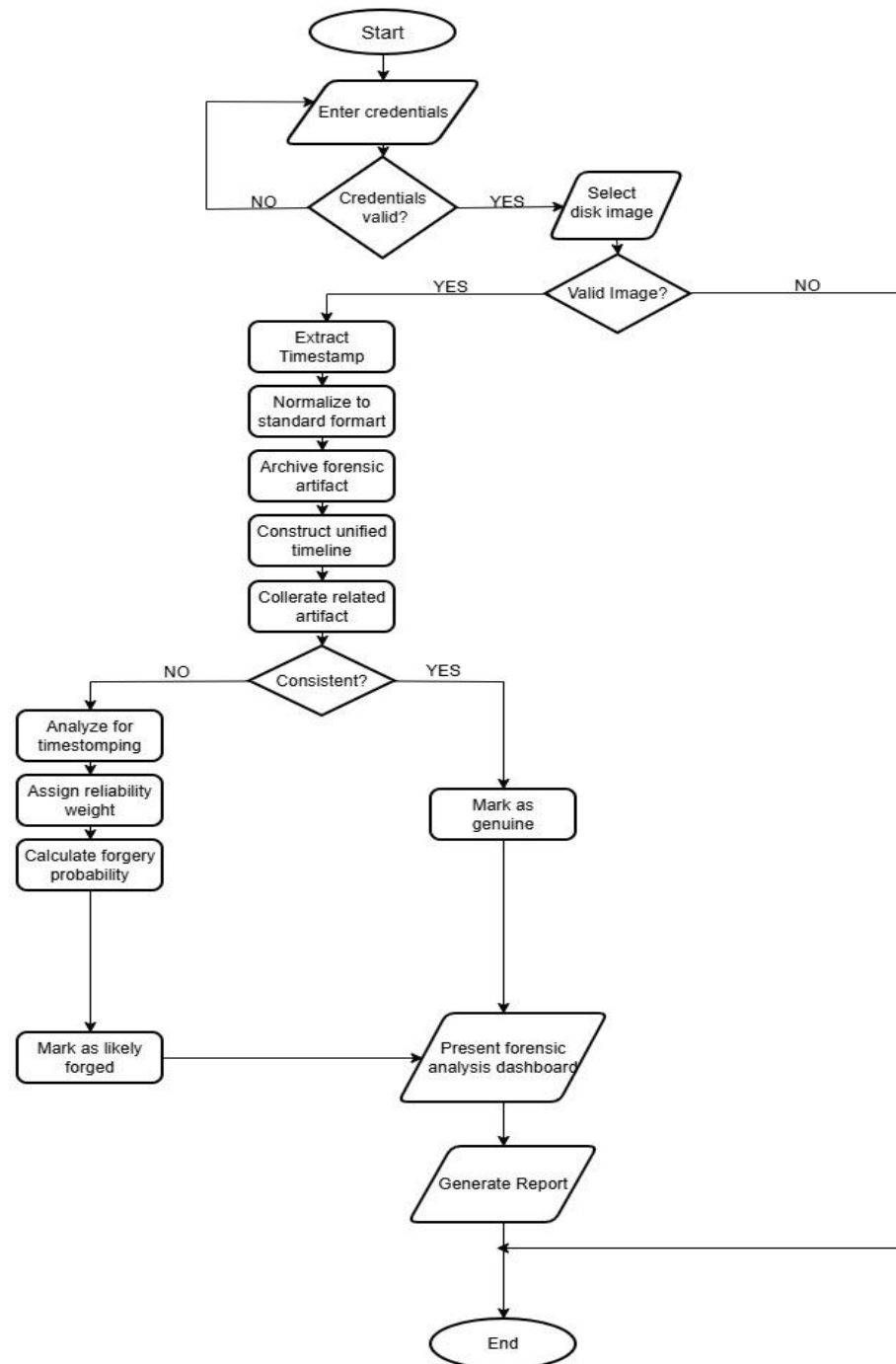


Figure 7 : General Flow chart

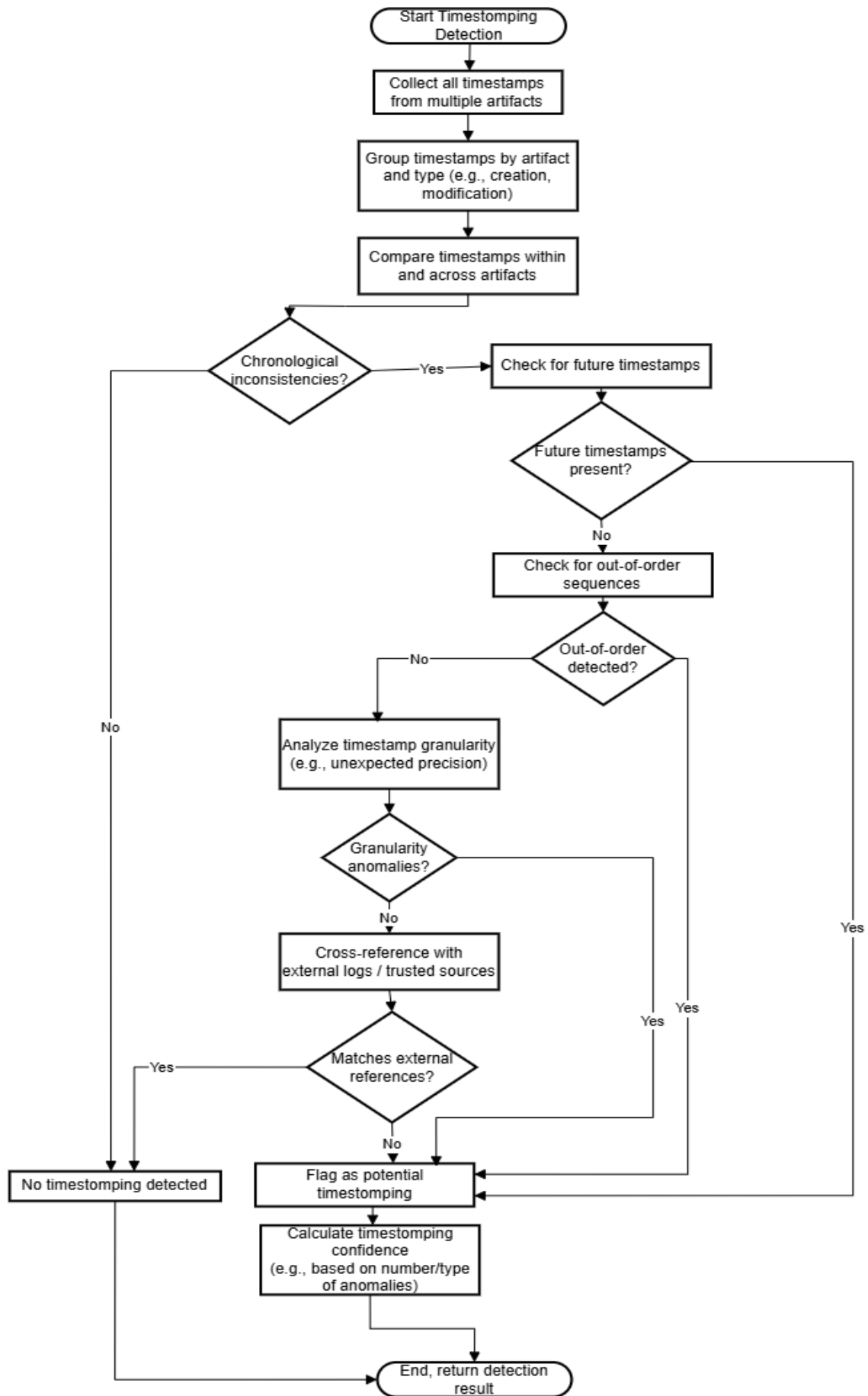


Figure 8 : Forgery Flow Chart

REFERENCES

- [1] B. Carrier, *File System Forensic Analysis*, Addison-Wesley, 2005.
- [2] J. Metz, “The Sleuth Kit and Autopsy: Open-Source Digital Forensics Tools,” 2023.
- [3] M. Jones and K. N. Levitt, “Detecting Timestamp Manipulation in File Systems,” *Digital Investigation Journal*, 2018.
- [4] A. Case and G. G. Richard III, “Digital Forensics and Incident Response: NTFS Artifact Analysis,” Elsevier, 2020.
- [5] S. L. Garfinkel, “Digital Forensics Research: The Next 10 Years,” *Digital Investigation*, vol. 7, pp. S64–S73, 2010.
- [6] E. Casey, *Digital Evidence and Computer Crime*, 3rd ed., Academic Press, 2011.
- [7] J. Sammons, *The Basics of Digital Forensics*, 2nd ed., Syngress, 2015.
- [8] Microsoft, “NTFS Technical Reference,” Microsoft Documentation, 2024.